# CLMSynth v0.6.7

Arootin Gharibian

August 12, 2026

**Abstract**

CLMSynth generates a label $L(x)$ and attaches it to an existing set of cluster IDs $c(x)$ (ground truth or algorithm output), with the relationship between $L$ and $c$ specified by a configuration file rather than measured afterward. A synthetic dataset generated with this program can be used to benchmark and evaluate cluster and label matching under user-definable parameters: well-defined clusters with random class assignments, and balanced or imbalanced clusters with varying degrees of cluster-label matching. Unlike existing clustering benchmarks that implicitly equate clusters with classes, the generator treats cluster formation and class assignment as independent processes with configurable matching outcomes. Each dataset instance includes ground-truth cluster membership, class labels, and metrics to enable reproducible, comparative benchmarking of cluster-label matching.

## License Summary

# 1 Introduction

Existing synthetic datasets for clustering performance measurement assume equivalence between latent cluster structure and assigned class labels. Cluster–label matched dataset synthesizer addresses the natural occurrence of classes within latent clusters by treating cluster generation and class assignment as independent, configurable, and explicitly defined through cluster-label matching. This design supports multiple benchmarks, including non-clusterable data with strong class separability, well-defined clusters with randomly assigned classes, and both balanced and imbalanced clusters. For each configuration, the output is a dataset of labels attached to the cluster membership, alongside summary diagnostics characterizing cluster-label matching (CLM) measures. By generating and systematically comparing reproducible CLM scenarios, this resource complements existing clustering benchmarks and supports evaluation of methods that study the relationships between clustering structure and class labels.

## 1.1 Expected Outcomes

Expected Outcomes are assessed by the degree to which a feature subset confirms or supports known classes or labels regardless of clustering performance (the extent to which the feature subset induces distinct, stable groups in the data). Cluster-label matching (CLM) is measured as the feature subset's effectiveness in predicting ground-truth class labels. These together characterize outcomes associated with a selected feature subset, evaluated in terms of both its induced cluster structure and its alignment with class labels. Each scenario describes how a given feature subset performs across clustering, classification, and interpretability.

## 1.2 Reference Implementation

The complete reference implementation, together with the pipeline scripts, configuration templates, and worked examples, is maintained in the project repository: `https://github.com/A-Gharibian/CLMSynth`. The source code is the authoritative description of the engine's behavior; this manual documents the configuration schema and the high-level design rationale.

### 1.2.1 Core CLM generator steps

1. **Validate**, structural checks fail fast: `num_classes` and the data's own cluster count $K$ must each lie in $[1, 64]$, a coarse backstop rather than a reliability boundary); `perfect` requires $M = K$; `single` requires `single_match`, $M \geq 2$, $K \geq 2$; `target_metric` is only compatible with `single`/`custom`, and `scope: pair` additionally requires `type: mcc` and `matching_mode: single`; any requested spatial placement requires per-point coordinates; unknown modes raise errors.

2. **Resolve proportions** $p$ is uniform if `balanced`; otherwise explicit `proportions` take precedence, with `skew_rule` as the fallback. Converted to integer counts via largest-remainder rounding.

3. **Matching mode** $\rightarrow$ alignment rules:

   - `random`, a random permutation of the fixed multiset holding each label $\ell_i$ exactly $m_i$ times, rather than an independent draw from $p$ per point: the realized counts equal $m_i$, and the permutation consults neither $c(x)$ nor position, so independence from $c(x)$ holds by construction rather than expectation.
   - `perfect`, bijection $\pi$ : cluster $\rightarrow$ label in sorted order; label counts are assigned to the paired cluster sizes; $L(x) = \pi(c(x))$.
   - `single`, one target pair $(k^*, \ell^*)$ aligned at full recall; all other points are assigned by spillover.
   - `custom`, an `assignment_matrix` of rules, each a `recall_target` fraction of one label's point budget into a chosen set of clusters (enabling surjective, partial, and overlapping).

4. **Allocation & feasibility**, each rule is split across its clusters (`split_rule`); exceeding cluster capacity raises an infeasibility error stating the maximum feasible recall. Remaining cluster capacity is filled by `spillover_rule`. (`proportional_to_marginal` preserves the target proportions).

5. **Target metric**, in both scopes, the solver varies only the recall level; every setting above is held fixed across probes, so noise structure changes the solved recall rather than being applied after it. With `scope: global` (the default), one recall level is solved for the whole partition by a coarse grid scan; with `scope: pair`, the single-pair MCC of the `single_match` pair is inverted in closed form and no search runs. A target above the configuration's reachable ceiling terminates at the closest achievable value.

The engine measures CLM, and a solution that converged internally can still land outside tolerance, most visibly at small $N$, where spillover placement is a large share of the outcome.
Three CLM measures are available: `clustering_mcc` is the `scikit-learn` multiclass Gorodkin's $R_K$, made permutation-invariant by Hungarian matching of the cluster–label contingency table; it is the value reported and solved for under `scope: global`. `clustering_mcc_pair` is the $2 \times 2$ Matthews $\phi$ of one cluster against one label, which needs no matching and is the quantity `scope: pair` targets. `clustering_ari` is the adjusted Rand index (ARI).
The global $R_K$/ARI between an $M$-label and a $K$-cluster partition has no closed form, which is why `scope: global` is solved numerically. The compact closed-form $TP^\star$ expression of the companion article inverts the *single* $2 \times 2$ MCC exactly; it is not the multiclass quantity, but it is not merely illustrative either, it is precisely what `scope: pair` evaluates, which is why that mode requires `type: mcc` and `matching_mode: single` and reaches its target without a search.
The adjusted Rand index measures agreement over *pairs* of points rather than through any correspondence between label and cluster identifiers, and is chance-corrected so that independent partitions score 0. Because it is a function of the contingency table's cell and marginal pair counts alone, it needs no alignment step: permuting the values of the generated label leaves it unchanged, it is symmetric in its two arguments, and it is defined for any $M$ and $K$. This is what distinguishes it from $R_K$, which is meaningful only after the Hungarian alignment `clustering_mcc`. Each of these three CLM measures answers a different question and must be chosen based on the goal of the dataset.

6. **Spatial placement** Configures optionally, within each cluster, *which* points receive each label is chosen by weighted sampling without replacement, with weights derived from normalized distance to the cluster centroid, with`profile`: `linear/step/exponential`; and `favors`: `steepness` for exponential or `core/boundary` for other profiles. The contingency table, and therefore the target CLM outcome, is fixed by stages 2–4, so spatial structure can be varied at a constant metric value.

# 2 Configuration file definition

The `clm_label_engine` is the core module for label synthesis. The configuration parameters for the core module are defined as follows:

```
clm_label:
  num_classes: M                    # 1 <= M <= 64; the data's
    own K is capped the
                                    # same way (CLM-126 / CLM
   -127)
  balance: balanced | unbalanced    # balanced: uniform 1/M,
   proportions ignored
  proportions: [...]                # unbalanced: used
   directly when given. Must have
                                    # exactly M entries (CLM
   -121) summing to 1 (CLM-106)
  skew_rule: geometric | dominant_minority | dirichlet
                                    # unbalanced fallback when
    no proportions
  skew_params:
    ratio: 0.5                      # geometric: p_i ~ ratio^i
    dominant_index: 0               # dominant_minority
    dominant_share: 0.5             # dominant_minority
    alpha: 1.0                      # dirichlet: Dirichlet(
   alpha,...,alpha)
  matching_mode: perfect | single | random | custom
  single_match: {cluster: k*, label: l*}    # required if
   matching_mode: single
  assignment_matrix:                          # required if
   matching_mode: custom
    - {label: l, clusters: [k1, ...], recall_target: r}
  split_rule: proportional_to_size | equal   # how a rule's
   mass splits over its clusters
  spillover_rule: proportional_to_marginal | uniform |
   concentrated
  concentrated_labels: [l, ...]     # spillover_rule:
   concentrated only. A LIST of
                                    # integer ids in 0..M-1 (
   CLM-128); a bare number
                                    # is read as a range, not
   a one-element list
                                    # (default: the single
   largest label)
  target_metric:                    # optional (single/custom
   only)
    type: mcc | ari
    value: 0.7
    scope: global | pair            # global (default): one
   recall solved
```

```
                                    #   numerically for the
  whole partition.
                                    # pair: exact closed form
  for the
                                    #   single_match pair;
  requires type: mcc
                                    #   and matching_mode:
  single.
   tolerance: 0.01                  # both scopes: how far the
   DELIVERED
                                    #   labeling may sit from
  the target
                                    #   before CLM-309/CLM-310
   is raised
   max_iter: 40                     # global only, bisection
  budget; the pair
                                    #   scope inverts exactly
  and never iterates
competing_noise:                    # optional (single/custom)
  : structured noise.
                                    # Converts a share of ONE
  cluster's UNCLAIMED
                                    # points to one competing
  label. Bypasses the
                                    # target proportions by
  design (CLM-304).
   - cluster: k
     label: l_comp
     share: 1.0                     # fraction of that cluster
  's leftover points
     favors: boundary | core | random
centroid_dependence:
  enabled: true | false
  profile: linear | exponential | step
  favors: core | boundary
  steepness: 3.0                    # exponential only
```

Spatial placement (`centroid_dependence`, or a `competing_noise` entry favoring `core`/`boundary`) requires per-point feature vectors. The complete catalog of coded errors and warnings is the troubleshooting reference.

## Usage

The input variables required for the generator (`X` and `c`; cluster centroids are derived internally from `X`) are expected to be supplied by external clustering benchmarks. CLMSynth is designed to work with documented datasets, such as the Gagolewski clustering benchmark suite (`clustbench`) or synthetic datasets generated via MDCGen (`mdcgen`). Also, synthetic data via an offline feature generator (`fabricated_data`) and direct import via `byoc` (*bring-your-own-clusters* are possible. The same `clm_label` configuration applies.
The package installs three entry points: `clmsynth` runs the batch pipeline over a configuration YAML, `clmsynth-config` renders a configuration from an upstream payload, and `clmsynth-wizard` generates one interactively. The engine can be called directly as a library:

```
from clmsynth.clm_label_engine import generate_clm_labels

config = {
    "num_classes": 3,
    "balance": "unbalanced",
    "proportions": [0.5, 0.3, 0.2],
    "matching_mode": "custom",
    "assignment_matrix": [
        {"clusters": [1, 2], "label": 0, "recall_target":
   0.9},
    ],
    "split_rule": "proportional_to_size",
    "spillover_rule": "proportional_to_marginal",
    "centroid_dependence": {
        "enabled": True,
        "profile": "exponential",
        "favors": "core",
        "steepness": 4.0,
    },
}

L = generate_clm_labels(
    cluster_labels=c,    # shape (N,)
    coords=X,            # shape (N, D)
    cfg=config,
    seed=42,
)  # -> pandas Series, values in {0, ..., M-1}
```

# Revision History

| Revision | Date | Author(s) | Description |
|---|---|---|---|
| 0.1 | 2026-07-07 | AG | Initial release: CLM label engine, `[CLM-###]` diagnostics, configuration wizard, MCC/ARI evaluation. |
| 0.2 | 2026-07-15 | AG | Internal, unpublished. |
| 0.3 | 2026-07-31 | AG | `clustering_mcc_pair` added to the API; `fabricated_data` cluster ids changed from strings to integers. |
| 0.4 | 2026-07-31 | AG | The renderer can now emit `target_metric`, `competing_noise`, `concentrated_labels`, `skew_params` and `steepness`; `mdcgen` source repaired. |
| 0.5 | 2026-07-31 | AG | Research release: `CITATION.cff`, `[CLM-1xx]` configuration errors abort the run. |
| 0.6.2 | 2026-08-02 | AG | First public release. Label-space and pair-scope guards: `[CLM-121]`, `[CLM-128]`, `[CLM-129]`, `[CLM-130]`, `[CLM-153]` and `[CLM-310]` reject configurations that previously produced silently incorrect labels; documentation revised throughout |
| 0.6.3 | 2026-08-08 | AG | Correctness release: `[CLM-131]` validates `skew_params`; `scope: pair` now honors `tolerance`; run folders are created atomically; `[CLM-104]`/`[CLM-105]` report per dataset instead of aborting a batch. |
| 0.6.4 | 2026-08-09 | AG | Tooling and import hygiene: the target-metric search and the final generation now share one random stream, so a solved value is the value delivered; `byoc` imports are validated against stated requirements (see the uncoded-rejections section of the troubleshooting reference. |

| | | | |
|---|---|---|---|
| 0.6.5 | 2026-08-10 | AG | The test suite becomes part of the repository. |
| 0.6.6 | 2026-08-11 | AG | A patch release |
| 0.6.7 | 2026-08-12 | AG | CLI-Wizard update and improvement |

## Licenses for Third-Party Libraries

Python is distributed under the Python Software Foundation License V.2. This project relies on several open-source third-party Python libraries, including `NumPy`, `Pandas`, `SciPy`, `scikit-learn`, `Matplotlib`, `Seaborn`, and `PyYAML`. Optionally `faker`, `mdcgenpy`, and `pyivm`. These packages are distributed under their respective licenses (BSD 3-Clause, MIT, and Apache 2.0). Refer to the official documentation of each package or the project repository for full third-party licensing details.

# 3 References


- Gagolewski, M. (2020). *A framework for benchmarking clustering algorithms*. `https://clustering-benchmarks.gagolewski.com/weave/suite-v1.html`
- Iglesias, F., Zseby, T., Ferreira, D. *et al.* (2019). *MDCGen: A Multidimensional Dataset Generator for Clustering*. Journal of Classification. `https://link.springer.com/article/10.1007/s00357-019-9312-3`
- Jeon, H., Aupetit, M., Shin, D., Cho, A., Park, S., and Seo, J. (2025). *Measuring the Validity of Clustering Validation Datasets*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 47(6), 5045-5058. `https://doi.org/10.1109/TPAMI.2025.3548011`

# 4 Test Data

| Cfg | Configuration | Expected | Actual |
|---|---|---|---|
| A1 | perfect, $M{=}K{=}2$ | bijection; MCC $\approx 1$ | MCC = 1.000, ARI = 1.000 |
| A2 | random, $M{=}4$ | no cluster info; MCC $\approx 0$ | MCC = 0.018, ARI = $-0.000$ |
| A3 | single (cluster 1, $M{=}4$) | feasible here ($512 < 1024$); partial alignment | MCC = 0.279, ARI = 0.166 |
| A4 | custom, explicit recalls (0.8/0.5/0.3) | feasible; counts $[1024, 614, 410]$ | MCC = 0.527, ARI = 0.393 |
| B1 | custom + solve MCC= 0.6 | reachable here; solver hits it within tol. | solved MCC = 0.608 (target 0.6; **met**) |
| B2 | custom + solve ARI= 0.4 | reachable here; solver hits it within tol. | solved ARI = 0.407 (target 0.4; **met**) |
| B3 | custom + solve MCC= 0.99, max_iter 15 | unreachable; non-convergence + delivered-value check | solved MCC = 0.718 (**not met**) |
| C1 | single largest + centroid linear/core | MCC $\approx$ A3 (placement in 4-D) | MCC = 0.279, ARI = 0.166 |
| C2 | single largest + centroid exp/boundary | same contingency as C1 | MCC = 0.279, ARI = 0.166 |
| C3 | single largest + centroid step/core | same contingency as C1 | MCC = 0.279, ARI = 0.166 |
| D1 | single (cluster 1, $M{=}2$) | fills cluster exactly; MCC $\approx 1$ | MCC = 1.000, ARI = 1.000 |

Table 2: Cluster–label matching outcomes on `g2mg_4_20` ($N = 2048$, 4-D, $K = 2$ clusters of 1024 points), with two large clusters. The CLM target of single-match configurations A3 and D1 is feasible. Configurations A4 and B1–B3 share one rule set (labels sized $[1024, 614, 410]$, one rule per label); B1 and B2 request values possible within their configuration (solver within the 0.01 tolerance), whereas B3, a 3-label partition of a 2-cluster, requests MCC = 0.99, which is not possible. The solver returns its closest feasible value. C1–C3 differ only in spatial placement and are identical.

# Configuration Troubleshooting Reference

CLMSynth v0.6.7

August 12, 2026

This reference documents every configuration condition that is either an incompatibility or an ignored setting, as well as the diagnostic message of the CLMSynth label engine. It is intended to help users *plan* a valid configuration and *interpret* what the engine does with a specific configuration.

**How the code works.** The identifiers below are a documentation index for this manual; the core label synthesizer engine raises standard Python exceptions with messages banded by type:

- `1xx` **ValueError** invalid value (`100`–`110`) or **Incompatible** option (`111`–`149`) or **InfeasibleAllocationError**, Infeasible outcome (`150`–`169`).
- `2xx` **KeyError** a required configuration key is missing (message is just the key name).
- `3xx` **Warning** non-fatal; logged via `logging.warning`, execution continues.

## 1 Configuration option compatibility at a glance

| Option | perfect | single | random | custom |
|---|---|---|---|---|
| `proportions` / `balance` `skew_rule` | ○ | ✓ | ✓ | ✓ |
| `single_match` | – | ✓ req | – | – |
| `assignment_matrix` | – | – | – | ✓ req |
| `split_rule` | – | ○ | – | ✓ |
| `spillover_rule` | ○ | ✓ | – | ✓ |
| `competing_noise` | ○ | ✓ | × | ✓ |
| `target_metric` | × | ✓ | × | ✓ |
| `target_metric.scope: pair` | – | ✓ | – | × |
| `centroid_dependence` | ○ | ✓ | ○ | ✓ |

Table 1: Option × matching_mode compatibility: ✓ used / required (**ValueError** or **KeyError** if required but not provided), – not read / not applicable, ○ accepted but ignored or no visible effect (with **Warning**), × not allowed (raises a **ValueError** of type *Incompatible*).

Not every configuration rejection is diagnostic `CLM`. There are conditions that check whether the *data* that were imported by the configuration is a usable clustering at all. They do not carry a code, and the reader should not look for one in the message.

# 2 Requirements & Restrictions

## 2.1 Requirements (planning)

The user needs to provide, through a configuration file, a required set of configuration settings (Table 2). It is necessary to plan the required configuration before executing the label synthesizer; an independent CLI wizard module (`config_wizard`) provides step-by-step generation of the file.

| Setting | Requires |
|---|---|
| `num_classes` ($M$) | Integer in $[1, 64]$ ($\geq 2$ under `single`); the dataset's own cluster count $K$ is capped the same way. A coarse safety backstop (126/127), not a validated reliability boundary, internal testing already finds results unreliable well below this, at $K \gtrsim 15$ or $M \gtrsim 20$. |
| `matching_mode` | One of `perfect`, `single`, `random`, `custom`. |
| `balance: unbalanced` | Either explicit `proportions` *or* a `skew_rule`. Explicit proportions need exactly `num_classes` entries (121) summing to 1 (106). Rules have defaults, so `skew_params` may be omitted. |
| `skew_params` | `skew_params` for its type: `geometric`→`ratio`; `dominant_minority`→`dominant_index`,`dominant_share`; `dirichlet`→`alpha`. |
| `matching_mode:` | |
| `perfect` | `num_classes` == number of clusters ($M = K$); a bijection needs one label per cluster. |
| `single` | $M \geq 2$, $K \geq 2$, and `single_match:{cluster,label}`; the cluster must exist and the label must be in $0..M-1$. Refer to `target_metric.scope` |
| `custom` | `assignment_matrix` with $\geq 1$ row; each row's `label`$\in 0..M-1$ and `clusters` exist; each row needs `recall_target` *unless* `target_metric` is set. |
| `target_metric` | `matching_mode` $\in$ {`single`, `custom`}; `type`$\in$`{mcc,ari}`; `value`$\in [-1, 1]$. |
| `target_metric.scope` | `"global"` (default) or `"pair"`. `pair` additionally requires `type: mcc` and `matching_mode: single`, and is met in closed form (`tolerance`/`max_iter` are not read). |
| spatial placement | Per-point feature vectors (an $(N, d)$ `coords` array) whenever `centroid_dependence` is enabled or a `competing_noise` entry favors `core`/`boundary`; labels-only configs must omit them. |
| `competing_noise` | `matching_mode` $\in$ {`single`, `custom`}; each entry: `cluster` exists, `label`$\in 0..M-1$, `share`$\in [0, 1]$, `favors`$\in$`{core,boundary,random}`. Needs *unclaimed* (leftover) points to have any effect. |
| `centroid_dependence` | `profile`$\in$`{linear,exponential,step}`; `favors`$\in$`{core,boundary}`, matched exactly and case-sensitively (129); `steepness` only for `exponential`. |
| `concentrated_labels` | Under `spillover_rule: concentrated` only: a *list* of integer label ids, each $\in 0..M-1$ (128). A bare number is not a shorthand for a one-element list. Omit the key to take the default, the single largest label. |

Table 2: List of configuration a user is required to plan and provide to the software

## 2.2 Incompatibilities

Incompatibilities are configurations in which every individual setting is valid, but the combination is not: the requested features are mutually exclusive so that no assignment can satisfy both at once.

| Feature | Not allowed when | Code | Reason & fix |
|---|---|---|---|
| `target_metric` | `matching_mode = perfect` | 111 | Perfect is a fixed bijection (MCC= 1); there is no free `recall_target` to search. Use `single`/`custom`, or drop `target_metric`. |
| `target_metric` | `matching_mode = random` | 114 | Random ignores clusters entirely; there is no agreement level to dial. Use `single`/`custom`. |
| `competing_noise` | `matching_mode = random` | 115 | There is no aligned label for a competing label to compete with. Use `single`/`custom`. |
| `target_metric.scope: pair` | `type = ari` | 123 | The single-pair MCC inverts in closed form; ARI has no such inverse. Use `type: mcc`, or `scope: global`. |
| `target_metric.scope: pair` | `matching_mode` $\neq$ `single` | 124 | The target pair is the `single_match` (cluster, label). Use `scope: global` for `custom`. |
| `spatial placement` | `features` missing or empty | 125 | Centroid distances need complete feature values. Pass an $(N, d)$ feature array, or disable `centroid_dependence`. |
| `centroid_dependence.favors` | not `core`/`boundary` | 129 | Matching is case-sensitive and exact. Any other value, including a capitalization slip (`Core`) or a typo (`boundary`), is only read when `enabled: true`. |
| `target_metric.scope: pair` | `spillover_rule` is `uniform`; or `concentrated` targeting the pair's own label (or left to its default); or a `competing_noise` entry emitting that label | 130 | The closed form sizes $\ell^*$ so that *all* of it sits inside $k^*$, and there is no search to correct a miss. Anything that can emit $\ell^*$ elsewhere silently delivers a different coefficient. Use `proportional_to_marginal`, whose leftover pool for $\ell^*$ is empty by construction, or `scope: global`. |

Table 3: Incompatibilities between several configurations, which are reported as 1xx value errors as invalid configuration, but cannot be diagnosed by inspecting any single field.

## 3 Ignored & Irrelevant Settings (expectations)

If the requirements are met with no incompatibilities in configuration (Section 4.1–4.3), all other mode×option mismatches are not considered as errors and are ignored with a warning (see Section 4.4 for a complete reference).

| Setting | Ignored / irrelevant when | Effect |
|---|---|---|
| `proportions` | `balance = balanced` | Replaced by a uniform $1/M$ split. Warns 301. |
| `proportions`, `balance`, `skew_rule` | `matching_mode = perfect` | Label counts are forced to each paired cluster's size. Warns 302. |
| `skew_rule` | `balance = unbalanced` *and* `proportions` given | Explicit `proportions` take precedence; `skew_rule` is the fallback only. |
| per-row `recall_target` | `target_metric` is set | Overridden by the single solved global $\alpha$. Warns 303. |
| `centroid_dependence` | `matching_mode = perfect` | Runs but has no visible effect: each cluster is 100% one label, so placement cannot discriminate. |
| `centroid_dependence` | `matching_mode = random` | Placement never runs: random returns before the placement stage. The *coords guard* still applies, though, being checked ahead of that early return: `enabled: true` without feature vectors raises 125 even here. |
| `competing_noise` | `matching_mode = perfect` | No-op: zero unclaimed capacity to convert. Warns 305 per entry. |
| `split_rule` | a rule targets a single cluster | Short-circuited (the point mass goes to that one cluster directly). |
| `split_rule`, `spillover_rule` | `matching_mode = random` | No per-cluster allocation step exists at all in random mode. |
| `single_match` | `matching_mode` $\neq$ `single` | Not read. |
| `assignment_matrix` | `matching_mode` $\neq$ `custom` | Not read. |
| `steepness` | `profile` $\neq$ `exponential` | Unused (only the exponential weight uses it). |
| `concentrated_labels` | `spillover_rule` $\neq$ `concentrated` | Unused, and not validated in this case; default target is the single largest label. Under `concentrated` it *is* validated (128). |
| `tolerance`, `max_iter` | `target_metric.scope: pair` | Not read; the pair target is met in closed form, not by search. |

Table 4: Ignored configurations and their effect.

# 4 Error and Warnings Reference

## 4.1 Value Error (invalid configuration) 1xx

| Code | Trigger | Reason | Mitigation |
|---|---|---|---|
| 101 | `matching_mode` is none of the four modes | Unrecognised mode string | Use `perfect`, `single`, `random`, or `custom`. |
| 102 | `perfect` with $M \neq K$ | A bijection needs exactly one label per cluster | Set `num_classes` to the true cluster count, or switch to `custom`. |
| 103 | `single` with $M < 2$ or $K < 2$ | Single needs a target label *and* at least one other, over $\geq 2$ clusters | Raise `num_classes`; use data with $\geq 2$ clusters. |
| 104 | `single_match.label` / a row `label` not an integer in $0..M-1$ | The label id does not exist | Use a value in $0..$`num_classes`$-1$. |
| 105 | `single_match.cluster` / a row `cluster` not among the data's ids | Typo, or wrong id *type* (`byoc` ids are whatever your CSV column holds, and may be strings) | Use an id from the data; match the type (int vs string). |
| 106 | explicit `proportions` do not sum to 1 (tol. $10^{-6}$) | Not a probability distribution | Rescale so the entries sum to 1. |
| 107 | `skew_rule` not in {`geometric`,`dominant_minority`,`dirichlet`} | Unrecognised skew rule | Use one of the three, or give `proportions` directly. |
| 108 | `split_rule` not in {`proportional_to_size`,`equal`} | Unrecognised split rule | Use one of the two. |
| 109 | `spillover_rule` not in {`proportional_to_marginal`,`uniform`,`concentrated`} | Unrecognised spillover rule | Use one of the three. |
| 110 | `profile` not in {`linear`,`exponential`,`step`} | Unrecognised placement profile | Use one of the three. |
| 111 | `target_metric` under `perfect` | No free `recall_target` to solve for | Use `single`/`custom`, or drop `target_metric`. |
| 112 | `target_metric.type` not `mcc`/`ari` | Only these two metrics are solvable | Set `type: mcc` or `ari`. |
| 113 | `target_metric.value` outside $[-1, 1]$ | MCC/ARI live in $[-1, 1]$ | Choose a value in range. |
| 114 | `target_metric` under `random` | No cluster–label structure to dial | Use `single`/`custom`. |
| 115 | `competing_noise` under `random` | No aligned label to compete with | Use `single`/`custom`. |
| 116 | `competing_noise.favors` not `core`/`boundary`/`random` | Unrecognised placement side | Use one of the three. |
| 117 | `competing_noise.share` outside $[0, 1]$ | Share is a fraction of leftover points | Choose `share` in $[0, 1]$. |
| 118 | `competing_noise.label` outside $0..M-1$ | The competing label id does not exist | Use a valid label id. |
| 119 | `competing_noise.cluster` not among the data's ids | Typo or wrong id type | Use an existing cluster id. |
| 120 | `target_metric`: every probed $\alpha \in [0, 1]$ is infeasible | The rule clusters cannot hold any recall level (Table 6) | Enlarge the target clusters, lower `num_classes`, or adjust `proportions`. |

| Code | Trigger | Reason | Mitigation |
|---|---|---|---|
| 121 | `proportions` length $\neq$ `num_classes` | A longer list silently enlarged the label space: the surplus entries became label ids that `num_classes` never declared, written into the dataset through `uniform`/`concentrated` spillover. A shorter list left labels unallocated | provided exactly `num_classes` proportions. Not raised under `balance: balanced`, where `proportions` are ignored outright (301). |
| 122 | `target_metric.scope` not `pair`/`global` | Unrecognised scope string | Use `pair` or `global`. |
| 123 | `scope: pair` with `type: ari` | Only the single-pair MCC has a closed-form inverse | Use `type: mcc`, or `scope: global`. |
| 124 | `scope: pair` outside `single` mode | The pair is defined by `single_match` | Use `matching_mode: single`, or `scope: global`. |
| 125 | placement requested without feature vectors | Centroid distances need $(N, d)$ coordinates | Pass `coords`, or disable `centroid_dependence` and spatial `competing_noise`. |
| 126 | `num_classes` outside $[1, 64]$ | Coarse safety backstop against runaway/typo configs, not a validated reliability limit (see Table 2) | Use a value in $[1, 64]$; results are already unreliable well before this ceiling. |
| 127 | the dataset's cluster count $K > 64$ | Same backstop, applied to the data rather than the config | Regroup or subsample clusters to $K \leq 64$. |
| 128 | `concentrated_labels` is not a list of integers in $0..M-1$ | The list is drawn from directly and written into the label column, so an unchecked value puts a label in the dataset that `num_classes` never declared. A bare number is read as a *range* (scattering the remainder over that many labels, the opposite of `concentrated`); a non-integer is truncated on write | Give a list of existing label ids, e.g. `[0]`. Checked before the `target_metric` solver, so a solved score can never be computed over a label that does not exist. |
| 129 | `centroid_dependence` mode `.favors` not `core`/`boundary` | Matched exactly and case-sensitively; any other value, including `Core` or a typo, was silently treated as `boundary` | Use lowercase `core` or `boundary`. Only read when `enabled: true`. |
| 130 | `scope: pair` combined with a spillover or noise setting that can place the target label outside its cluster | The closed-form inversion assumes every point of $\ell^*$ lies in $k^*$; `uniform` spillover, `concentrated` onto $\ell^*$, and `competing_noise` emitting $\ell^*$ all break that assumption. | Use `spillover_rule: proportional_to_marginal`, keep `competing_noise` off $\ell^*$, or use `scope: global`. |
| 131 | `skew_params` out of range for the chosen `skew_rule` | Unchecked values mostly did not crash: they produced *negative* label counts that still summed to $N$, so largest-remainder rounding was satisfied and the run delivered a labeling nobody requested. The remaining cases raised uncoded `ZeroDivisionError`/`IndexError` | Use `geometric` `ratio` $\geq$ 0; `dominant_minority` `dominant_share` $\in [0, 1]$ with integer `dominant_index` $\in 0..M-1$ and $M \geq 2$; `dirichlet` `alpha` $> 0$. Only read when `balance` is not `balanced` *and* no `proportions` are given; omit `skew_params` for the defaults. |

Table 5: Complete list of invalid configurations.

Invalid configurations in Table 5 are covered by the `config_wizard` CLI utility; thus, to generate valid configurations, the wizard is the recommended tool.

## 4.2 Infeasible Allocation Error 15x

| Code | Trigger | Reason | Mitigation |
|---|---|---|---|
| 150 | A single rule: $\lceil r \cdot m_\ell \rceil >$ capacity of its clusters | The label's requested budget exceeds the target clusters' size | Lower `recall_target`, add clusters to the rule, or shrink the label via `proportions`. (Message states the max feasible recall.) |
| 151 | Rules jointly claim more than a cluster's size | Overlapping rules oversubscribe one cluster | Lower the competing `recall_target`s or spread them across more clusters. |
| 152 | `competing_noise` entries for one cluster claim more than its unclaimed points | Too much structured noise demanded on one cluster | Lower the `share` value(s). |
| 153 | Several `assignment_matrix` rules name the same label and jointly claim more than its budget | A rule's `recall_target` is a fraction of the label's *whole* budget, so repeating a label across rules adds those fractions. The overshoot silently broke the requested proportions: `proportional_to_marginal` spillover clips its pool at zero and cannot pull the count back, | Lower the recalls so they sum to $\leq 1.0$, or write one rule listing all the clusters and let `split_rule` divide the budget. |

Table 6: Infeasible Allocation Error, *Under `target_metric`*, codes 150/151/152/153 are caught internally and treated as "this $\alpha$ is infeasible" during the search; only 120 (no $\alpha$ works at all) surfaces to the user.

All errors in range 1xx have example configurations documented in the project repository; issues must be addressed after checking these invalid configurations and reported only if previously not covered.

## 4.3 Key Error (a required key is missing) 2xx

| Code | Missing key | Required when & fix |
|---|---|---|
| 201 | `num_classes` | Always. Add `num_classes: M`. |
| 202 | `matching_mode` | Always. Add `matching_mode: <mode>`. |
| 203 | `skew_rule` | When `balance: unbalanced` and no `proportions`. Add a `skew_rule` or explicit `proportions`. |
| 204 | `single_match` | When `matching_mode: single`. Add `single_match:{cluster,label}`. |
| 205 | `cluster` / `label` inside `single_match` | Both sub-keys are required. Add the missing one. |
| 206 | `assignment_matrix` | When `matching_mode: custom`. Add at least one rule. |
| 207 | `label` / `clusters` inside a rule | Every rule needs both. Add the missing one. |
| 208 | `recall_target` inside a rule | In `custom` *without* `target_metric`. Add `recall_target`, or add `target_metric` to solve it globally. |
| 209 | `value` inside `target_metric` | When `target_metric` is present. Add `value: <target>`. |

Table 7: Supplementary information for the context of KeyErrors (e.g. `KeyError: 'single_match'`).

## 4.4 Warnings (non-fatal) 3xx

| Code | Trigger | Recommendation |
|---|---|---|
| 301 | `balance: balanced` *and* `proportions` given | Expected. To use your `proportions`, set `balance: unbalanced`; otherwise remove them to silence the warning. |
| 302 | `perfect` ignores `proportions`/`balance`/`skew_rule` | Expected. Perfect fixes counts to cluster sizes; drop those keys to make intent clear. |
| 303 | `target_metric` present with per-row `recall_target`s | Expected. The global $\alpha$ overrides them; you may delete `recall_target` from the rows. |
| 304 | `competing_noise` active | Expected: achieved counts deviate from `proportions`. If you need exact proportions, do not use `competing_noise`. |
| 305 | A `competing_noise` entry hits a cluster with no unclaimed points | The cluster is fully claimed (or `share` rounds to 0): Lower a rule's `recall_target`, or target a different cluster. |
| 306 | `target_metric` did not converge within tolerance | The target likely exceeds the structural ceiling (e.g. $M \ll K$). The closest feasible value is returned; lower the target, raise `max_iter`/`tolerance`, or change `proportions`/clusters. |
| 307 | `scope: pair` target outside the pair's reachable range | The request is clamped to the nearest reachable value; the message reports the range. Choose a target within it, or change the pair. |
| 308 | `scope: pair` resizes the target label | Expected: the label is sized as a subset of its cluster so the pair MCC meets the target exactly; an explicit proportion for that label is overridden. |
| 309 | the CLM achieved `target_metric` is outside tolerance | The search scores candidates on a fixed probe stream. At the same time, the output is generated on the run's own stream, so a solve that converged internally can still land outside tolerance (most often at small $N$, where spillover placement dominates the outcome). **The achieved value in the message is authoritative, not the requested one.** To close the gap: raise `tolerance`, use a larger dataset, or use `scope: pair` (closed form, no search). |
| 310 | `scope: pair`: the delivered pair MCC is outside the request | Backstop for the closed form, the counterpart of 309 for the global solver. The combinations known to break the construction are rejected up front (130), so this firing means something else moved $\ell^*$ out of $k^*$; the message reports how many of its points did. **The achieved value is authoritative, not the requested one.** |

Table 8: Warning codes, which indicate scenarios of successfully executed configuration, but with a different outcome than the user would expect.

## 4.5 Uncoded Rejections (data import)

Not every rejection is a CLM diagnostic. The banded codes above describe the *cluster-label matching model*: what a configuration asks the engine to do, and whether that is coherent. The conditions in this section describe something prior to it, whether the *data* that was handed to the engine is a usable clustering at all.

This section is expected to grow as more input paths gain checks. Currently, it covers one: **Bring-Your-Own-Clusters** (`data_source: byoc`).

### BYOC input requirements

BYOC is an *import* path, not a generator. The premise is that the user has already clustered a subset of their features with their own algorithm (k-means, fuzzy c-means, or any other) and is bringing the result: the feature subset in which the clustering was computed, plus one column holding each point's cluster. The requirements in the following are derived from that premise rather than from the CLM engine.

Every requirement is checked in a single pass, so a file with several problems reports all of them at once rather than one per attempt. Any failure rejects that dataset; under a multi-dataset configuration, the remaining datasets are unaffected.

| Requirement | Rejected when | Why |
|---|---|---|
| Non-empty file | No data rows, or no columns | A header alone is not a clustering. |
| Distinct column names | Any name appears twice | `pandas` renames the second to `name.1`, so the column meant is ambiguous, and which of the two originals it holds depends on their order in the file. |
| No reserved names | A column is named `Cohort_Class`, `GroundTruth_*`, `Cluster_n` or `Label_n` | The first two are consumed as ground truth and would silently disappear as features; the last two are written into the output, producing a duplicate column in the result CSV. |
| Declared cluster column | `cluster_column` missing, not a single name, or absent from the file | The ground-truth partition must be named explicitly; it is never guessed. |
| Complete cluster column | Any missing value | Every point belongs to a cluster. A blank would become a cluster of its own and change $K$. |
| At least two clusters | The cluster column holds one distinct value | A single partition has no structure for a label to agree or disagree with. |
| Minimum cluster size | Any cluster holds fewer than 3 points | A cluster of one or two points is a stray rather than a cluster. The engine's arithmetic accepts it, which is the problem: the run would proceed on a partition the user did not intend. |
| Numeric features | Any non-cluster column is non-numeric | Every column other than the cluster column is treated as a feature, and features define the geometry the clustering was computed in. |
| Complete features | Any missing value in a feature column | The clustering cannot have been computed from missing values, and centroid distances would evaluate to `NaN` without complaint. |

Table 9: BYOC import requirements. These reject a dataset before any label generation; they are not CLM codes, because they concern the data rather than the cluster-label matching model.

**What is *not* restricted.** Cluster identifiers may be integers or strings, in any script; if `single_match` or `assignment_matrix` references them, the type must match. The names of the feature columns are carried verbatim to the output CSV and the plot axes. Repeated identical rows are kept, since they are additional points rather than a data error, and removing them would change the cluster sizes on which the matching arithmetic is built.

**Known limitation.** There is currently no way to carry a column that is neither a feature nor the cluster, an additional label or "other tag" that should travel with the data without entering the geometry. Every non-cluster column is a feature, which is why all of them must be numeric. A declaration for such columns is planned.